

QUESTIONS 10 / 10 completed

BST- Mirroring

BST- MIRRORING

Write a C program to mirror a Binary Tree and display the inorder traversal before and after mirroring. Consider a graphical application where tree structures need to be reflected horizontally for visualization purposes.
Sample Input: Nodes:

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*in(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->d=x;t->l=t->r=NULL;
10    }else if(x<t->d)t->l=in(t->l,x);
11    else t->r=in(t->r,x);
12    return t;
13 }
14
15 void print(struct node*t){
16     if(t){print(t->l);printf("%d ",t->d);print(t->r);}
17 }
18
19 void mirror(struct node*t){
20     struct node*p;
21     if(t){
```

INPUT

7
10 5 15 3 7 12 18

QUESTIONS 10 / 10 completed

Binary Tree-Inorder ▾

BINARY TREE-INORDER REVERSAL

Write a C program to create a Binary Tree and perform Recursive Inorder Traversal. Consider an organizational hierarchy management system where employee records are stored in a binary tree structure. The program should

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int x) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = x;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int x) {
17     if (root == NULL)
18         return create(x);
19
20     if (x < root->data)
21         root->left = insert(root->left, x);
```

INPUT

50 30 70 20 40 60 80

QUESTIONS 10 / 10 completed

BT- Rec. Preorder T ▼

BT- REC. PREORDER TRAVERSE

Write a C program to create a Binary Tree and perform Recursive Preorder Traversal. Consider a file backup application where directories and subdirectories are represented as a tree structure. The program should display the order in which folders

PROGRAM EDITOR

C ▼

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int x) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = x;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int x) {
17     if (root == NULL)
18         return create(x);
19
20     if (x < root->data)
21         root->left = insert(root->left, x);
```

INPUT

10 5 15 3 7 12 18

QUESTIONS 10 / 10 completed

BT- Rec. Postorder ▾

BT- REC. POSTORDER TRAVERSE

Write a C program to create a Binary Tree and perform Recursive Postorder Traversal. Consider a software uninstallation utility where dependent components must be removed before the parent application. The program should display the deletion

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int x) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = x;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int x) {
17     if (root == NULL)
18         return create(x);
19
20     if (x < root->data)
21         root->left = insert(root->left, x);
```

INPUT

25 15 35 10 20 30 40

QUESTIONS 10 / 10 completed

BST- In,Pre,Post Tra ▾

BST- IN,PRE,POST TRAVERSE

Write a C program to create a Binary Search Tree (BST) and perform Inorder, Preorder, and Postorder Traversals. Consider a student record management system where roll numbers are stored in a BST and different traversals are used for various reporting

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int x) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = x;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int x) {
17     if (root == NULL)
18         return create(x);
19
20     if (x < root->data)
21         root->left = insert(root->left, x);
```

INPUT

45 25 65 15 35 55 75

QUESTIONS 10 / 10 completed

BST- Search Element

BST- SEARCH ELEMENT

Write a C program to insert elements into a Binary Search Tree and search for a specific element. Consider an online product catalog where product IDs are stored in a BST for efficient retrieval. The program should indicate whether a given product ID exists

PROGRAM EDITOR

C



Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int data;struct node *l,*r;};
5
6 struct node* insert(struct node* t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->data=x;t->l=t->r=NULL;
10    }else if(x<t->data)t->l=insert(t->l,x);
11    else t->r=insert(t->r,x);
12    return t;
13 }
14
15 int search(struct node*t,int x){
16     if(!t)return 0;
17     if(t->data==x)return 1;
18     return x<t->data?search(t->l,x):search(t->r,x);
19 }
20
21 int main(){
```

INPUT

```
7
50 30 70 20 40 60 80
60
```

QUESTIONS 10 / 10 completed

BST- Finding Height ▾

BST- FINDING HEIGHT

Write a C program to find the height of a Binary Tree. Consider a network routing structure where devices are arranged hierarchically and the tree height represents the maximum communication depth. The program should determine the height ▾

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*in(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->d=x;t->l=t->r=NULL;
10    }else if(x<t->d)t->l=in(t->l,x);
11    else t->r=in(t->r,x);
12    return t;
13 }
14
15 int h(struct node*t){
16     if(!t)return 0;
17     int a=h(t->l),b=h(t->r);
18     return (a>b?a:b)+1;
19 }
20
21 int main(){
```

INPUT

7
1 2 3 4 5 6 7

QUESTIONS 10 / 10 completed

BST- Count Nodes ▾

BST- COUNT NODES

Write a C program to count the total number of nodes, leaf nodes, and non-leaf nodes in a Binary Tree. Consider a company's organizational structure where leaf nodes represent employees without subordinates and non-leaf nodes represent

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*in(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->d=x;t->l=t->r=NULL;
10    }else if(x<t->d)t->l=in(t->l,x);
11    else t->r=in(t->r,x);
12    return t;
13 }
14
15 int count(struct node*t,int *leaf){
16     if(!t)return 0;
17     if(!t->l&&!t->r)(*leaf)++;
18     return 1+count(t->l,leaf)+count(t->r,leaf);
19 }
20
21
```

INPUT

7
50 30 70 20 40 60 80

QUESTIONS 10 / 10 completed

BST- Min, Max Val

BST- MIN, MAX VALUES

Write a C program to find the minimum and maximum values in a Binary Search Tree. Consider a sales database where transaction amounts are stored in a BST and management wants to identify the smallest and largest transactions. Sample

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*in(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->d=x;t->l=t->r=NULL;
10    }else if(x<t->d)t->l=in(t->l,x);
11    else t->r=in(t->r,x);
12    return t;
13 }
14
15 int main(){
16     struct node*t=NULL,*p;
17     int n,x,i,min,max;
18     scanf("%d",&n);
19     for(i=0;i<n;i++){scanf("%d",&x);t=in(t,x);}
20     p=t;while(p->l)p=p->l;min=p->d;
21     p=t;while(p->r)p=p->r;max=p->d;
```

INPUT

```
7
100 50 150 25 75 125
175
```

QUESTIONS 10 / 10 completed

BST- Deletion

BST- DELETION

Write a C program to delete a node from a Binary Search Tree and display the tree using inorder traversal after deletion. Consider a library management system where discontinued book IDs must be removed from the catalog.

Sample Input: Nodes:

PROGRAM EDITOR

C



Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*in(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->d=x;t->l=t->r=NULL;
10    }else if(x<t->d)t->l=in(t->l,x);
11    else t->r=in(t->r,x);
12    return t;
13 }
14
15 struct node*del(struct node*t,int x){
16     struct node*p;
17     if(!t)return t;
18     if(x<t->d)t->l=del(t->l,x);
19     else if(x>t->d)t->r=del(t->r,x);
20     else{
21         if(!t->l){p=t->r;free(t);return p;}
```

INPUT

```
7
50 30 70 20 40 60 80
30
```